typica v0.1.0 · Typst >= 0.15.1

Monochrome HTML blog bundles with native MathML, right-margin sidenotes, mathematical environments, and RSS / Atom syndication.

import	`#import "@preview/typica:0.1.0": *`
scaffold	`typst init @preview/typica:0.1.0 my-blog`
bundle	`typst compile --features html,bundle --format bundle site.typ public`
license	`MIT` (`src/`) · `MIT-0` (`template/`, `examples/`)

1 Architecture & Compilation Targets 1
2 Project Layout & Optional Post Folders 1
3 Library Reference (Generated with `tidy`) 2
3.1 Site Bundler (`src/site.typ`) 2
3.2 Post Template & Assets (`src/post.typ`) 3
3.3 Environments, Code & Layout (`src/components.typ`) 5
3.4 Margin Sidenotes (`src/notes.typ`) 9
3.5 Theme, Fonts & Stylesheet (`src/theme.typ`) 9
3.6 Syndication Feeds (`src/feeds.typ`) 11

01 Architecture & Compilation Targets

`typica` compiles a pure Typst project directly into a complete static blog without external static-site generators, client-side JavaScript frameworks, or manual MathML post-processing[1].

Every post remains a valid standalone `.typ` document while `site.typ` mounts posts inline and emits the entire web root in a single compiler invocation:

[1] A 14-line event listener handles code-block copy buttons; every layout, sidenote toggle, and dark/light rule is pure HTML + CSS.

MODE	COMMAND	OUTPUT
Full site (`bundle`)	`typst compile --features html,bundle -f bundle site.typ public`	`index.html`, `404.html`, `posts/<slug>/index.html`, assets, `css/style.css`, `rss.xml`, `atom.xml`
Single post (`html`)	`typst compile --features html -f html posts/fixed-points.typ`	Self-contained HTML5 post with inlined `style.css`
Live editor (`paged`)	`typst watch posts/fixed-points.typ`	Instant `height: auto` paged preview with right-margin sidenotes
Archival print	Browser `Ctrl+P` on any HTML post	Two-column `@media print` layout preserving right-margin sidenotes on paper

02 Project Layout & Optional Post Folders

Posts without external files live directly as flat `.typ` files (`posts/foo.typ`). When a post needs co-located images, SVGs, or downloadable files, place it in its own directory (`posts/bar/index.typ`) alongside those assets:

```text
my-blog/
├── site.typ                  # Homepage (<site>), feeds, and mounted posts
└── posts/
    ├── fixed-points.typ      # Flat post -> posts/fixed-points/index.html
    └── automaton/            # Optional folder when co-locating assets
        ├── index.typ         # -> posts/automaton/index.html
        └── dfa.svg           # -> posts/automaton/dfa.svg
```

Cross-references work across the entire bundle using standard Typst labels:

- `@site` (or `@site[home]`) links back to `index.html` from any post.
- `@fixed-points` links to `posts/fixed-points/index.html` using the post's formatted title.
- `@kleene-fp` jumps directly to section `= ... <kleene-fp>` inside whichever post defines it.
- `@rss[rss]` and `@atom[atom]` link to `rss.xml` and `atom.xml` on the homepage.

03 Library Reference (Generated with `tidy`)

The reference sections below are parsed directly from `typica`'s source modules (`src/*.typ`) using `@preview/tidy:0.4.3`.

3.1 Site Bundler (`src/site.typ`)

```
post-list(show-summary: true) -> content
```

Render a newest-first `<ul class="entries">` list of all non-draft, non-unlisted posts in the active bundle using `query-all-posts()`.

PARAMETERS.

`show-summary` `bool` `= true`

Include each post's `summary` (or `subtitle` fallback) beneath its title row.

```
site(
  title: auto,
  description: none,
  url: "https://example.com",
  author: none,
  posts-dir: "posts",
  pages: (),
  assets: (),
  not-found-page: true,
  rss-path: "rss.xml",
  atom-path: "atom.xml",
  theme: (:),
  body,
) -> content
```

Top-level site bundler (`#show: site.with(...)`).

Renders `site.typ`'s body as `index.html` (`<site>`), emits every mounted `#post(include "posts/foo.typ") <foo>` (or `"posts/foo/index.typ"`) as `<posts-dir>/foo/index.html` (`<foo>`) alongside any co-located post assets, and generates `css/style.css` (`<stylesheet>`), `404.html`, `rss.xml` (`<rss>`), and `atom.xml` (`<atom>`). Feed `<link rel="alternate">` discovery tags are emitted exclusively on `index.html`.

PARAMETERS.

`title` `auto` `content` `str` = auto
Site title (`auto` infers from the first `= ...` heading in `body`, or pass any string / rich content such as `something` or `$sin(x)$` — rendered automatically as the homepage `<h1>` if `body` omits a top-level heading, and serialized to plain text for `<title>`, `404.html`, and RSS / Atom feeds).

`description` `content` `str` `none` = none
Short site description for `<meta name="description">` and feed subtitles.

`url` `str` = "https://example.com"
Canonical origin URL (without trailing slash) used to construct absolute feed links.

`author` `content` `str` `none` = none
Default author inherited by posts that omit `author` and emitted in Atom `<author>`.

`posts-dir` `str` = "posts"
Output directory prefix for emitted post pages (`posts/<slug>/index.html`).

`pages` `array` = ()
Additional standalone HTML pages `(slug: "about", title: [About], body: [...])`.

`assets` `array` = ()
Site-wide static assets `(path: "robots.txt", data: bytes(...), label: none)`.

`not-found-page` `bool` = true
Emit `404.html` styled with the active site palette and typography.

`rss-path` `str` `none` = "rss.xml"
Output bundle path for the RSS 2.0 feed, or `none` to disable RSS generation.

`atom-path` `str` `none` = "atom.xml"
Output bundle path for the Atom 1.0 feed, or `none` to disable Atom generation.

`theme` `dictionary` = (:)
Theme overrides merged onto `default-theme` across both HTML CSS and paged previews.

`body` `content`
Homepage markup (`= Heading`, `#links-row(...)`, and mounted `#post(...) <slug>` entries).

3.2 Post Template & Assets (`src/post.typ`)

`post-asset`(path, data) -> `content`

Declare a co-located file asset inline inside a post body (`#post-asset("dfa.svg", read("dfa.svg", encoding: none)) <dfa>`). In bundle mode, `site` emits the file alongside the post's `index.html` (`posts/<slug>/<path>`).

PARAMETERS.

`path` `str`
Relative target filename inside the post's output directory.

`data` `bytes` `str`
Raw binary `bytes` (`read(..., encoding: none)`) or UTF-8 string payload.

`query-all-posts`() -> `array`

Query all non-draft, non-unlisted `<typica-post-meta>` records across an HTML bundle, sorted newest-first with resolved `slug`, `words`, `minutes`, `doc-loc`, and `html-path`. Must be called inside `context`.

```
post(
  title: auto,
  subtitle: none,
  created: none,
  modified: none,
  date: none,
  updated: none,
  author: none,
  monograph: none,
  tags: (),
  summary: none,
  assets: (),
  slug: auto,
  path: none,
  home-text: "/..",
  wpm: 200,
  show-wordcount: true,
  draft: false,
  unlisted: false,
  numbered-headings: true,
  footnotes-as-sidenotes: true,
  theme: (:),
  page-width: auto,
  margin: auto,
  body,
) -> content
```

Dual-purpose post template and inline homepage mounter:

- In a post file (`posts/foo.typ` or `posts/foo/index.typ`):
  `#show: post.with(title: [...], author: "...", created: ..., modified: ...)`
- In `site.typ`: `#post(include "posts/foo.typ") <foo>` (or `"posts/foo/index.typ"` when co-locating assets)

PARAMETERS.

`title` `auto` `content` `str` = auto
Post title (`<h1>` in HTML, semibold heading in paged mode); `auto` detects an included post in `site.typ`.

`subtitle` `content` `str` `none` = none
Optional italic subtitle beneath `title` (also used as fallback summary).

`created` `datetime` `str` `none` = none
Creation date (`YYYY-MM-DD`); drives feed publication timestamps and newest-first sort order.

`modified` `datetime` `str` `none` = none
Last-modified date (`YYYY-MM-DD`); drives Atom `<updated>`.

`date` `datetime` `str` `none` = none
Alias for `created`.

`updated` `datetime` `str` `none` = none
Alias for `modified`.

`author` `content` `str` `none` = none
Post author; falls back to `site.with(author: ...)` inside a bundle.

`monograph` `content` `str` `none` = none
Optional EWD-style monograph code (e.g. `"EWD-02"`) rendered right beneath `author` in post frontmatter.

`tags` `array` = ()
Plain `#tag` strings rendered in `Commit Mono` and exported as RSS / Atom `<category>`.

`summary` `content` `str` `none` = none
Short abstract displayed on homepage entry cards and in RSS / Atom feeds.

`assets` `dictionary` `array` = ()
Co-located files (`("dfa.svg": read("dfa.svg", encoding: none))`) emitted into `posts/<slug>/`.

`slug` `auto` `str` = auto

Explicit slug override; defaults to the `<label>` attached to `#post(...) <slug>`.

path `str` `none` = none

Optional custom bundle output path when mounting in `site.typ` (defaults to `<posts-dir>/<slug>/index.html`).

home-text `content` `str` `none` = "/.."

Backlink label at the top of HTML post pages (automatically omitted in PDF and `@media print`).

wpm `int` = 200

Words-per-minute reading speed used to compute `reading` minutes.

show-wordcount `bool` = true

Display `words` and `reading` rows in the post frontmatter block.

draft `bool` = false

Exclude post from homepage listings and RSS / Atom feeds when `true`.

unlisted `bool` = false

Emit the HTML page at its URL, but omit it from homepage listings and feeds.

numbered-headings `bool` = true

Prefix top-level `=` sections with `01`, `02`, … in `Commit Mono`.

footnotes-as-sidenotes `bool` = true

Automatically route standard `#footnote[...]` calls into the right margin.

theme `dictionary` = (:)

Per-post theme overrides layered onto the active site theme.

page-width `auto` `length` = auto

Paged preview width override (`auto` uses `theme.page-width`).

margin `auto` `dictionary` = auto

Paged preview margin override (`auto` uses `theme.margin`).

body `content`

Post body markup (when used via `#show: post.with(...)`) or `include "posts/..."` content (when mounted in `site.typ`).

3.3 Environments, Code & Layout (`src/components.typ`)

```
grid-table(body, ..rest) -> content
```

Inner crosshair grid table built on `tablem` (`left` & `top` hairlines between cells).

PARAMETERS.

body `content`

Markdown-style pipe table content block.

..rest `any`

Additional arguments forwarded to `tablem`.

```
zebraw(
  ..args,
  highlight-color: auto,
  background-color: auto,
  comment-color: auto,
  lang-color: auto,
) -> content
```

Monochrome-framed code block wrapper around `@preview/zebraw:0.6.3` with native syntax highlighting, line numbers, line highlights, and HTML copy button.

PARAMETERS.

..args `any`

Code block and options forwarded to `@preview/zebraw`.

highlight-color `auto` `color` = auto

Highlighted line background (`auto` uses `theme.surface-2-light`).

background-color `auto` `color` = auto

Code block background (`auto` uses `theme.surface-1-light`).

comment-color `auto` `color` = auto
Line-comment annotation fill (`auto` uses `theme.surface-2-light`).
lang-color `auto` `color` = auto
Language badge fill (`auto` uses `theme.surface-2-light`).

```
sbox(
  title,
  ..sections,
  variant: "recipe",
  kind: none,
) -> content
```

Multi-section monochrome environment box with uppercase sans-serif `kind` label.

PARAMETERS.
title `content` `str` `none`
Box title (`none` or `[]` omits the underlined title).
..sections `content`
One or more body content blocks separated by dotted hairline rules.
variant `str` = "recipe"
Visual surface variant (`"definition"`, `"theorem"`, `"example"`, `"recipe"`).
kind `str` `none` = none
Uppercase sans-serif category label (e.g. `"Definition"`, `"Theorem"`).

```
definition(title, ..sections) -> content
```

Mathematical definition environment (`DEFINITION.`).

```
#definition[Pointed CPO][
  An $omega$-CPO $(D, subset.eq.sq)$ with least element $bot in D$.
]
```

DEFINITION. **Pointed CPO**
An ω-CPO $(D, \sqsubseteq)$ with least element $\bot \in D$.

PARAMETERS.
title `content` `str` `none`
Definition title.
..sections `content`
One or more body sections.

```
theorem(title, ..sections) -> content
```

Theorem environment (`THEOREM.`).

```
#theorem[Kleene Fixed Point][
  Every Scott-continuous map $F : D -> D$ has $"lfp"(F) = union.sq.big_(n >= 0) F^n (bot)$.
]
```

THEOREM. **Kleene Fixed Point**
Every Scott-continuous map $F : D \to D$ has $\mathrm{lfp}(F) = \bigsqcup_{n \geq 0} F^n(\bot)$.

PARAMETERS.
title `content` `str` `none`
Theorem title.
..sections `content`
One or more body sections.

```
lemma(title, ..sections) -> content
```

Lemma environment (`LEMMA.`).

proposition(title, ..sections) -> content

Proposition environment (`PROPOSITION.`).

example(title, ..sections) -> content

Example environment (`EXAMPLE.`).

recipe(title, ..sections) -> content

Recipe / procedure environment (`RECIPE.`).

remark(title, ..sections) -> content

Remark environment (`REMARK.`).

proof(title: [Proof], body) -> content

Proof block with subtle left rule and flush-right hollow Halmos square (`sym.square.stroked`).

```
#proof[
  By induction on $n$, $F^n (bot) subset.eq.sq e$ for every pre-fixed point $e$.
]
```

PROOF.
By induction on n, $F^n(\bot) \sqsubseteq e$ for every pre-fixed point e. □

PARAMETERS.

title content str = [Proof]
Uppercase sans-serif lead-in label (e.g. `[Proof]` or `[Proof sketch]`).

body content
Proof body.

callout(kind: "note", title: auto, body) -> content

Monochrome left-bordered callout (`note`, `tip`, `warning`, `important`).

PARAMETERS.

kind str = "note"
Callout kind (`"note"`, `"tip"`, `"warning"`, `"important"`).

title auto content str none = auto
Custom heading (`auto` uses `kind`, `none` omits heading).

body content
Callout content.

epigraph(source: none, body) -> content

Section-opening italic epigraph with optional right-aligned attribution.

```
#epigraph(source: [Edsger W. Dijkstra, EWD249])[
  Simplicity is prerequisite for reliability.
]
```

Simplicity is prerequisite for reliability.

– Edsger W. Dijkstra, EWD249

PARAMETERS.

source content str none = none

Optional attribution rendered flush-right in `Commit Mono`.

body `content`

Quotation body.

wide(body) -> content

Full-width container spanning both the prose column and the right sidenote margin. Automatically clears any active right-margin sidenotes in both HTML (`clear: right`) and paged mode (`@preview/marge` page state).

PARAMETERS.

body `content`

Wide table, figure, or listing body.

diagram(caption: none, body) -> content

Render arbitrary Typst graphics natively in paged previews and as inline `<svg>` in HTML (automatically inverted in dark mode).

PARAMETERS.

caption `content` `str` `none` = none

Optional figure caption.

body `content`

Typst drawing or diagram content.

kbd(key) -> content

Monochrome keyboard shortcut badge (`#kbd("Ctrl+P")`).

```
Press #kbd("Ctrl+P") for archival two-column print output.
```

```
Press Ctrl+P for archival two-column print output.
```

PARAMETERS.

key `content` `str`

Key label text.

links-row(..items) -> content

Horizontal `Commit Mono` link row separated by middle dots (`·`) for thin homepages.

PARAMETERS.

..items `content`

Link elements (e.g. `link("https://example.org")[example.org]`, `[@rss[rss]]`).

tablem function

Markdown-like pipe table (`@preview/tablem:0.3.0`).

three-line-table function

Classic academic three-line table (`@preview/tablem:0.3.0`).

codeblock function

Alias for `zebraw`.

hr `content`

Minimalist horizontal rule (`<hr>` in HTML, hairline rule in paged previews).

3.4 Margin Sidenotes (`src/notes.typ`)

```
sidenote(
  dy: 0pt,
  gap: 0.55em,
  padding: (left: 1.4em, right: 1.8cm),
  numbered: true,
  body,
) -> content
```

Place a numbered (or unnumbered) note in the right margin (`@preview/marge` in paged previews; `<span class="sidenote">` float on desktop HTML and `@media print`, collapsing into a tap-to-expand numbered card on viewports `<= 1100px`).

PARAMETERS.

dy `length` `ratio` `relative` = `0pt`
Vertical offset adjustment in paged mode.

gap `length` = `0.55em`
Minimum vertical separation between consecutive paged margin notes.

padding `length` `dictionary` = `(left: 1.4em, right: 1.8cm)`
Horizontal inset inside the right margin column in paged mode.

numbered `bool` = `true`
Display synchronized superscript numbers in prose and at the start of the margin note.

body `content`
Margin note content.

```
marginnote(dy: 0pt, gap: 0.55em, body) -> content
```

Place an unnumbered margin note alongside the current paragraph (`[+]` toggle on `<= 1100px`).

PARAMETERS.

dy `length` `ratio` `relative` = `0pt`
Vertical offset adjustment in paged mode.

gap `length` = `0.55em`
Minimum vertical separation between consecutive paged margin notes.

body `content`
Margin note content.

sn `function`

Short alias for `sidenote`.

mn `function`

Short alias for `marginnote`.

3.5 Theme, Fonts & Stylesheet (`src/theme.typ`)

Passing `theme: (...)` to `site.with(...)` or `post.with(...)` overrides any subset of `default-theme` across **both** the generated HTML stylesheet (`css/style.css`) and paged Typst previews:

KEY	DEFAULT	ROLE
`paper-light` / `paper-dark`	`oklch(99%, 0, 0deg)` / `oklch(18%, 0, 0deg)`	Page background (`--bg`)
`text-light` / `text-dark`	`oklch(18%, 0, 0deg)` / `oklch(93%, 0, 0deg)`	Primary ink (`--fg`)
`muted-light` / `muted-dark`	`oklch(52%, 0, 0deg)` / `oklch(65%, 0, 0deg)`	Secondary metadata, captions, sidenotes (`--muted`)
`border-light` / `border-dark`	`oklch(89%, 0, 0deg)` / `oklch(29%, 0, 0deg)`	Hairline dividers & code borders (`--line`)
`frame-light` / `frame-dark`	`oklch(20%, 0, 0deg)` / `oklch(84%, 0, 0deg)`	Environment & `grid-table` outer frames (`--frame`)
`surface-1-*` / `surface-2-*`	`96.5%` & `93.5%` / `22%` & `26%`	Code background, box headers, line highlights
`serif-fonts`	`("STIX Two Text", "Libertinus Serif")`	Body prose, headings, and sidenotes (`--serif`)
`math-fonts`	`("STIX Two Math", "New Computer Modern Math")`	Equations & native browser MathML (`--math`)
`sans-fonts`	`("Helvetica Neue", "Helvetica", "Arial", "DejaVu Sans")`	Uppercase environment & proof labels (`--sans`)
`mono-fonts`	`("Commit Mono", "CommitMono", "DejaVu Sans Mono")`	Code listings, frontmatter, tags, links row (`--mono`)
`font-faces`	`auto`	Emits `@font-face` blocks for active default web fonts; pass custom CSS `str` or `none`
`font-size`, `page-width`, `margin`	`10.8pt`, `210mm`, `(left: 2.2cm, right: 7.4cm, ...)`	Paged preview typography & sidenote margin geometry

`resolve-theme`(overrides) -> `dictionary`

Merge a user `theme` override dictionary onto `default-theme`.

PARAMETERS.

`overrides` `dictionary` `none`

Partial theme dictionary overriding any subset of `default-theme` keys.

`active-theme`() -> `dictionary`

Resolve the active theme inside `context` (preferring any enclosing per-post `<typica-post-theme>` override over site-wide `<typica-site-cfg>`).

`build-stylesheet`(theme: default-theme) -> `str`

Render `src/style.css` substituting all `oklch(...)` palette tokens, `--serif` / `--math` / `--sans` / `--mono` font stacks, and `@font-face` declarations from `theme`.

PARAMETERS.
theme `dictionary` = default-theme
Theme dictionary (`default-theme` or user overrides).

```
default-theme dictionary
```

Default perceptual `oklch(L, 0, 0deg)` monochrome palette, unified *STIX Two* + *Helvetica Neue* + *Commit Mono* font stacks, and paged preview geometry.

3.6 Syndication Feeds (`src/feeds.typ`)

```
build-atom-feed(
  site-title: [Blog],
  site-subtitle: none,
  site-url: "https://example.com",
  author: none,
  atom-path: "atom.xml",
  posts: (),
) -> str
```

Generate an Atom 1.0 XML feed string via `@preview/exemel` (`atom-encode`).

PARAMETERS.
site-title `content` `str` = [Blog]
Feed title.
site-subtitle `content` `str` `none` = none
Optional Atom `<subtitle>`.
site-url `str` = "https://example.com"
Canonical site origin URL.
author `content` `str` `none` = none
Default feed author name.
atom-path `str` = "atom.xml"
Relative path to the Atom feed (`atom.xml`).
posts `array` = ()
Post records returned by `query-all-posts()`.

```
build-rss-feed(
  site-title: [Blog],
  site-subtitle: none,
  site-url: "https://example.com",
  rss-path: "rss.xml",
  posts: (),
) -> str
```

Generate an RSS 2.0 XML feed string via `@preview/exemel` (`rss-encode`).

PARAMETERS.
site-title `content` `str` = [Blog]
Channel `<title>`.
site-subtitle `content` `str` `none` = none
Channel `<description>` (falls back to `site-title` when `none`).
site-url `str` = "https://example.com"
Canonical site origin URL.
rss-path `str` = "rss.xml"
Relative path to the RSS feed (`rss.xml`) for `atom:link rel="self"`.
posts `array` = ()
Post records returned by `query-all-posts()`.